

Sesión 2: R, estructura del proyecto, limpieza de datos y visualización

Curso: Modelos CGE y Microsimulación aplicados al Sistema de Pensiones de
Capitalización Individual

MACROPOL: Economía Aplicada y Políticas Públicas

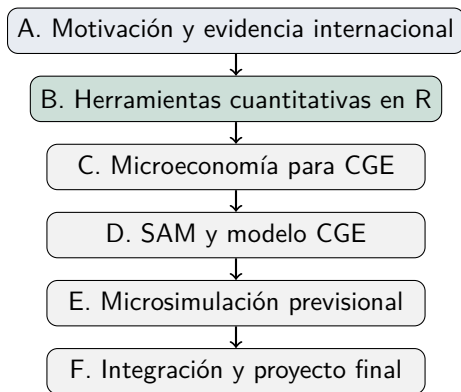
Meta práctica

Al finalizar la sesión, cada participante debe tener un proyecto en R organizado, reproducible y capaz de cargar, limpiar, transformar y visualizar una base preliminar para análisis previsional.

La sesión conecta tres piezas:

- ❶ **Estructura del proyecto:** cómo organizar archivos, datos y scripts.
- ❷ **Limpieza de datos:** cómo pasar de datos crudos a datos analíticos.
- ❸ **Visualización:** cómo producir evidencia clara para diagnóstico y política pública.

Dónde estamos en el roadmap del curso



Por qué iniciar con herramientas

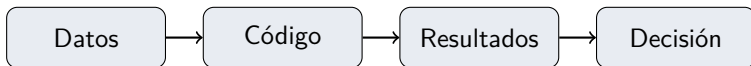
Idea central

Un modelo CGE o una microsimulación no comienzan con ecuaciones. Comienzan con un flujo de trabajo ordenado.

Errores comunes en proyectos cuantitativos:

- archivos duplicados o sin versión;
- bases modificadas manualmente;
- scripts que dependen de rutas locales;
- resultados que no pueden reproducirse;
- gráficos sin narrativa de política pública.

Toda simulación debe ser reproducible.



Si una persona no puede reconstruir el resultado desde los datos y el código, el resultado no está listo para política pública.

Estructura del proyecto

```
Curso_CGE_Pensiones/  
|-- data/  
|   |-- raw/           # datos originales, nunca se editan  
|   |-- intermediate/  # datos parcialmente procesados  
|   |-- clean/         # base analitica final  
|  
|-- R/  
|   |-- 00_setup.R  
|   |-- 01_importar_datos.R  
|   |-- 02_limpieza_datos.R  
|   |-- 03_visualizacion.R  
|   |-- 04_indicadores_previsionales.R  
|  
|-- outputs/  
|   |-- figures/  
|   |-- tables/  
|  
|-- docs/  
|-- README.md  
|-- Curso_CGE_Pensiones.Rproj
```

Los datos originales no se modifican

La carpeta data/raw/ debe contener la base tal como fue recibida. Todas las transformaciones se hacen mediante código.

Esto permite:

- auditar cambios;
- corregir errores sin reconstruir manualmente;
- actualizar bases cuando llegan nuevas rondas de información;
- documentar claramente el puente entre microdatos y resultados.

Script 00: configuración

```
# 00_setup.R
rm(list = ls())

paquetes <- c(
  "tidyverse", "data.table", "janitor", "here",
  "readxl", "scales", "skimr"
)

instalar <- paquetes[!paquetes %in% installed.packages()[, "Package"]]
if (length(instalar) > 0) install.packages(instalar)

invisible(lapply(paquetes, library, character.only = TRUE))

# Rutas principales
path_raw <- here::here("data", "raw")
path_clean <- here::here("data", "clean")
path_fig <- here::here("outputs", "figures")
path_tab <- here::here("outputs", "tables")
```


Datos para la sesión

Usaremos una base sintética con información individual inspirada en microdatos laborales y previsionales.

Variables principales:

- edad;
- sexo;
- nivel educativo;
- ingreso laboral;
- condición de formalidad;
- afiliación previsional;
- densidad de cotización;
- saldo acumulado;
- factor de expansión.

El objetivo no es estimar todavía el modelo final, sino aprender el flujo de trabajo.

Script 01: importar datos

```
# 01_importar_datos.R
source(here::here("R", "00_setup.R"))

personas_raw <- readr::read_csv(
  here::here("data", "raw", "personas_pensiones_sintetica.csv"),
  show_col_types = FALSE
)

# Estandarizar nombres de variables
personas <- personas_raw |>
  janitor::clean_names()

glimpse(personas)
skimr::skim(personas)
```

Diagnóstico inicial de calidad

Antes de limpiar, observamos.

Preguntas básicas:

- ¿Cuántas observaciones tiene la base?
- ¿Qué variables son numéricas, categóricas o texto?
- ¿Hay valores faltantes?
- ¿Existen edades, ingresos o saldos imposibles?
- ¿La variable de expansión está disponible?

En pensiones, los errores de datos pueden traducirse en diagnósticos equivocados de suficiencia y cobertura.

Script 02: reglas de limpieza

```
# 02_limpieza_datos.R
source(here::here("R", "00_setup.R"))
source(here::here("R", "01_importar_datos.R"))

personas_clean <- personas |>
  filter(edad >= 15, edad <= 100) |>
  mutate(
    ingreso_laboral = if_else(ingreso_laboral < 0, NA_real_, ingreso_laboral),
    saldo_afp       = if_else(saldo_afp < 0, NA_real_, saldo_afp),
    formal           = if_else(formal == 1, "Formal", "Informal"),
    afiliado_afp     = if_else(afiliado_afp == 1, "Afiliado", "No afiliado"),
    grupo_edad = case_when(
      edad < 30 ~ "15-29",
      edad < 45 ~ "30-44",
      edad < 60 ~ "45-59",
      TRUE      ~ "60+"
    )
  )
```

Crear variables analíticas

```
personas_clean <- personas_clean |>
  mutate(
    cotizante = if_else(formal == "Formal" & afiliado_afp == "Afiliado", 1, 0),
    aporte_anual_estimado = ingreso_laboral * 12 * 0.10 * cotizante,
    quintil_ingreso = ntile(ingreso_laboral, 5),
    tasa_aporte_efectiva = aporte_anual_estimado / (ingreso_laboral * 12)
  )

readr::write_csv(
  personas_clean,
  here::here("data", "clean", "personas_pensiones_clean.csv")
)
```

Variables puente hacia el modelo previsional

Variable	Uso en microsimulación	Uso en CGE
Ingreso laboral	salario inicial por individuo	masa salarial agregada
Formalidad	densidad de cotización	mercado laboral formal/informal
Edad	cohorte y horizonte de retiro	estructura demográfica
Afiliación AFP	cobertura previsional	contribuciones agregadas
Saldo AFP	riqueza previsional inicial	ahorro financiero
Factor expansión	agregación poblacional	cierre macro-micro

Del dato individual al indicador agregado

Ejemplo

La cobertura previsional no es solo el porcentaje de afiliados en la muestra. Debe calcularse usando factores de expansión.

$$Cobertura = \frac{\sum_i w_i \cdot Afiliado_i}{\sum_i w_i}$$

Donde w_i es el factor de expansión del individuo i .

Indicadores ponderados

```
indicadores <- personas_clean |>
  summarise(
    poblacion = sum(factor_expansion, na.rm = TRUE),
    cobertura_afp = weighted.mean(afiliado_afp == "Afiliado",
                                   factor_expansion, na.rm = TRUE),
    formalidad = weighted.mean(formal == "Formal",
                                factor_expansion, na.rm = TRUE),
    ingreso_promedio = weighted.mean(ingreso_laboral,
                                       factor_expansion, na.rm = TRUE),
    saldo_promedio = weighted.mean(saldo_afp,
                                     factor_expansion, na.rm = TRUE)
  )

indicadores
```


Un buen gráfico debe responder una pregunta concreta.

Ejemplos:

- ¿La cobertura aumenta con el ingreso?
- ¿La formalidad varía por edad?
- ¿Dónde se concentran los trabajadores sin AFP?
- ¿Qué grupos tendrían menor tasa de reemplazo?
- ¿Cómo cambia el diagnóstico antes y después de una reforma?

Visualizar no es decorar. Es revelar una relación económica.

Gráfico 1: cobertura por quintil

```
p1 <- personas_clean |>
  group_by(quintil_ingreso) |>
  summarise(
    cobertura = weighted.mean(afiliado_afp == "Afiliado",
                              factor_expansion, na.rm = TRUE),
    .groups = "drop"
  ) |>
  ggplot(aes(x = factor(quintil_ingreso), y = cobertura)) +
  geom_col() +
  scale_y_continuous(labels = scales::percent) +
  labs(
    title = "Cobertura AFP por quintil de ingreso",
    x = "Quintil de ingreso laboral",
    y = "Cobertura"
  ) +
  theme_minimal()
```

Gráfico 2: ingreso laboral por edad y formalidad

```
p2 <- personas_clean |>
  filter(!is.na(ingreso_laboral)) |>
  ggplot(aes(x = edad, y = ingreso_laboral)) +
  geom_point(alpha = 0.15) +
  geom_smooth(se = FALSE) +
  facet_wrap(~ formal) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Perfil edad-ingreso por formalidad",
    x = "Edad",
    y = "Ingreso laboral mensual"
  ) +
  theme_minimal()
```

Gráfico 3: saldo AFP por grupo de edad

```
p3 <- personas_clean |>
  filter(afiliado_afp == "Afiliado") |>
  ggplot(aes(x = grupo_edad, y = saldo_afp)) +
  geom_boxplot(outlier.alpha = 0.2) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Distribucion del saldo AFP por grupo de edad",
    x = "Grupo de edad",
    y = "Saldo acumulado"
  ) +
  theme_minimal()
```

Guardar resultados

```
ggsave(  
  filename = here::here("outputs", "figures", "cobertura_quintil.png"),  
  plot = p1, width = 8, height = 5, dpi = 300  
)  
  
ggsave(  
  filename = here::here("outputs", "figures", "perfil_edad_ingreso.png"),  
  plot = p2, width = 8, height = 5, dpi = 300  
)  
  
ggsave(  
  filename = here::here("outputs", "figures", "saldo_afp_edad.png"),  
  plot = p3, width = 8, height = 5, dpi = 300  
)
```

- ➊ Crear la estructura del proyecto.
- ➋ Cargar la base sintética de personas.
- ➌ Limpiar nombres y valores inválidos.
- ➍ Crear variables previsionales básicas.
- ➎ Calcular cobertura, formalidad e ingreso promedio.
- ➏ Construir tres gráficos de diagnóstico.
- ➐ Guardar la base limpia y las figuras.

Conexión con las siguientes sesiones

Lo construido hoy alimenta directamente los módulos posteriores.

Producto de hoy	Uso posterior
Base limpia	Microsimulación de tasas de reemplazo
Cobertura por grupo	Diagnóstico previsional inicial
Masa salarial	Contribuciones agregadas al CGE
Formalidad	Segmentación del mercado laboral
Saldo AFP	Stock inicial de riqueza previsional
Gráficos	Insumos para policy brief

Buenas prácticas desde el inicio

- Nunca editar manualmente bases crudas.
- Usar rutas relativas con `here()`.
- Separar importación, limpieza, análisis y visualización.
- Guardar resultados en carpetas estandarizadas.
- Nombrar scripts con orden numérico.
- Escribir comentarios breves, pero útiles.
- Pensar cada gráfico como respuesta a una pregunta.

Mensaje principal

Antes de construir un CGE o una microsimulación, necesitamos una cadena de trabajo confiable: datos ordenados, código reproducible e indicadores interpretables.

En la próxima sesión entraremos al repaso microeconómico que da sentido económico a las ecuaciones del modelo.